

32'h7FFF FFFF + 1

32'h7FFF_FFFF: 01111111111111111111111111111111

1 : 00000000000000000000000000000001

32'h7FFF_FFFF + 1 = 10000000000000000000000000000000

Z = 0, N = 1, C = 0, V = 1 (hubo overflow porque sumamos dos números positivos y el resultado es negativo)

32'h8000_0000-1:

32'h8000_0000: 10000000000000000000000000000000

1 : 00000000000000000000000000000001

Invertimos los bits de 1: 11111111111111111111111111111110

A este resultado le sumamos 1: 11111111111111111111111111111111 = 32'hFFFF_FFFF

Hacemos la suma entre 32'h8000_0000 y 32'hFFFF_FFFF:

10000000000000000000000000000000 + 11111111111111111111111111111111 =
01111111111111111111111111111111

Z = 0, N = 0, C = 1, V = 1

Ejercicio 2: Banco de registros

Instante/Acción	dout A	dout B	R2
Situación inicial	10	20	20
A cambia a índice 2, antes del flanco	10	20	20
Después del siguiente flanco	20	20	20
Después de escribir 99 en R2 por A	20	20	99
Después del siguiente flanco sin escritura	99	99	99

No se va a poder escribir R0, ya que al no entrar en los ifs del segundo flip-flop, rf[0] no recibirá ningún dato. Además, en el primer flip flop, cuando se lee el contenido previo en regA_idx y regB_idx, si el índice del registro es 0, el dato de salida será 0.

Ejercicio 3: Camino de datos

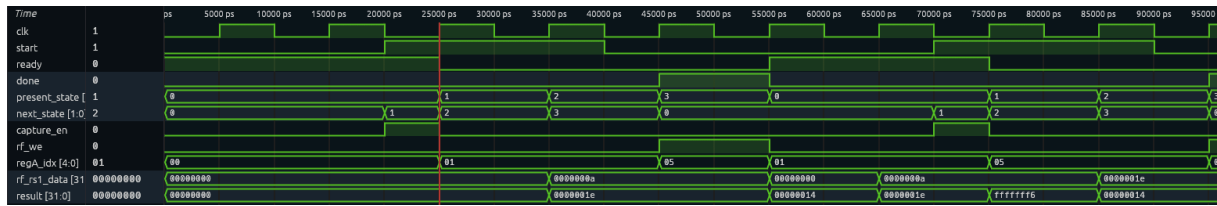
Los operandos se actualizan cada vez que cambien los dout de los registros A y B del reg_file. Esto sucede cuando reg_file termina de leer el contenido de los índices seleccionados. El operando A depende del índice indicado en rs1 o rd, y el operando B depende de rs2.

El destino se escribe cuando rf_we esta habilitado. Al estar habilitado, se toma como índice rd para el registro A, y tambien se habilita la escritura en ese mismo registro. Esto hace que en el siguiente flanco ascendente, se escriba el valor de dout del registro A (es cual es el resultado de la operacion realizada) en el registro indicado en el índice rd.

Como comentamos antes, el operando A depende del índice indicado en rs1 o rd, aun que no depende 100% de ellos, sino que tambien depende de la habilitacion o no de rf_we Como tambien dijimos antes, el operando A toma el valor (anterior, ya que se realiza una escritura al mismo tiempo) del registro indicado en rd cuando rf_we esta habilitado, pero cuando no lo esta, toma el valor del registro indicado en rs1. Este funcionamiento hace que rd no cambie inmediatamente por cada vez que se setee un nuevo valor de índice.

Ejercicio 6: Unidad de ejecución completa

Flanco de captura



Flanco de lectura



Flanco de escritura



present_state	regA_idx	result
FETCH_OPS	01	00000000
EXECUTE	01	0000001e
WRITEBACK	05	0000001e

El resultado se calcula inmediatamente luego de que la ALU reciba los valores de los operandos para realizar la operacion, quedando disponible para su uso. Luego en el siguiente flanco ascendente del clk, y con rf_we en 1, se escribe su valor en el registro destino (R5 en el ejemplo del ejercicio).